BSP Layers

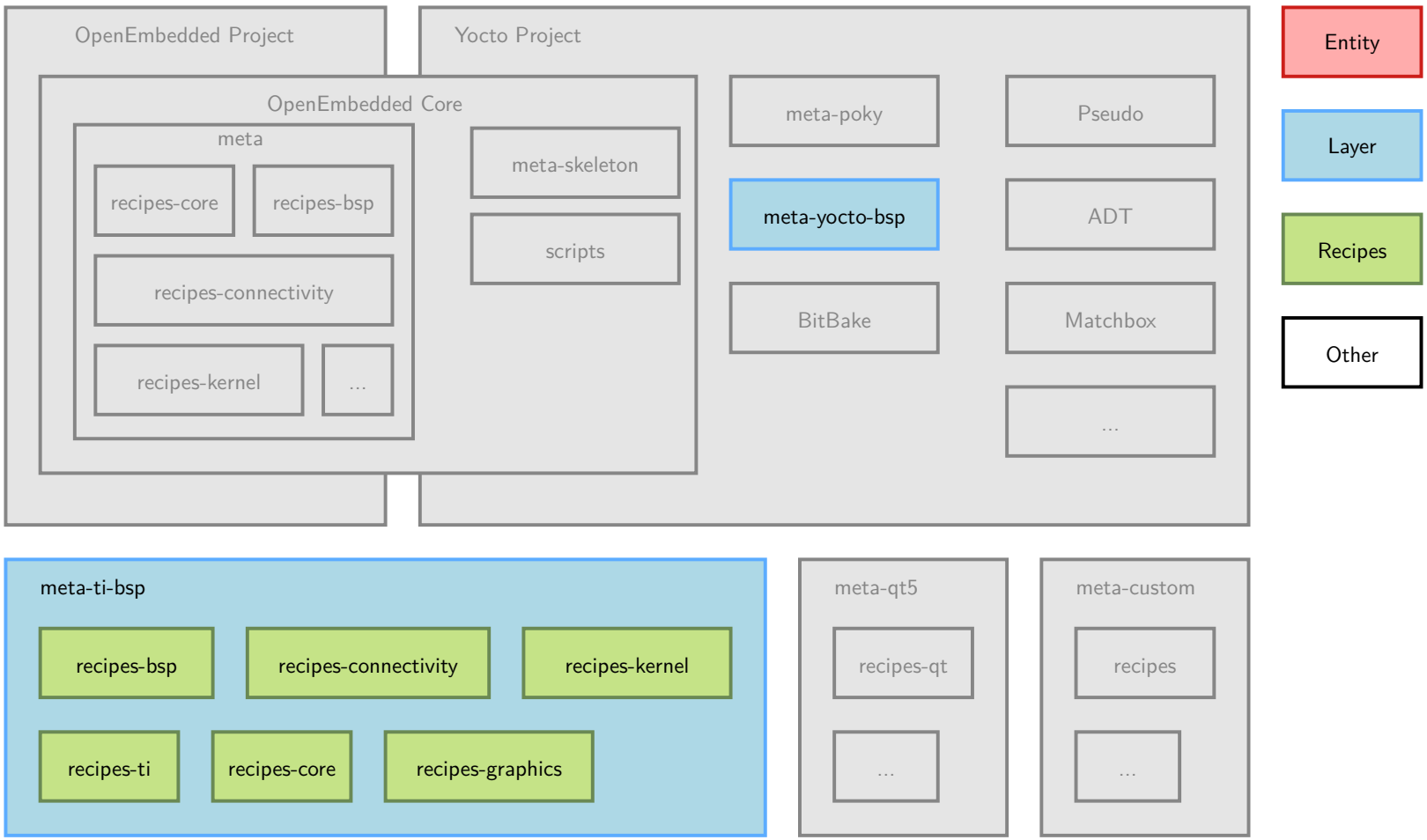


© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Introduction to BSP layers in the Yocto Project

BSP layers


OpenEmbedded Project
Yocto Project
OpenEmbedded Core
meta
recipes-core
recipes-bsp
recipes-connectivity
recipes-kernel
...
meta-skeleton
scripts
meta-poky
meta-yocto-bsp
BitBake
Pseudo
ADT
Matchbox
...
Entity
Layer
Recipes
Other
meta-ti-bsp
recipes-bsp
recipes-connectivity
recipes-kernel
recipes-ti
recipes-core
recipes-graphics
meta-qt5
recipes-qt
...
meta-custom
recipes
...



BSP layers overview

- BSP layers are a subset of the layers.
- They hold metadata with the purpose of supporting a specific class of hardware devices.
- They usually provide:
  - Hardware configuration files (`machines`)
  - Custom kernel and bootloader recipes and configurations
  - Modules and drivers to enable specific hardware features (e.g. multimedia accelerators)
  - Pre-built user binaries and firmware
- A good practice is to name it `meta-<bsp_name>`.
- Examples: `meta-ti-bsp`, `meta-st-stm32mp`.



Hardware configuration files



Overview 1/2

- A layer provides one machine file (hardware configuration file) per machine it supports.
- These configuration files are stored under `meta-<bsp_name>/conf/machine/*.conf`
- The file names correspond to the values set in the `MACHINE` configuration variable.
  - `meta-ti/meta-beagle/conf/machine/beaglebone.conf`
  - `MACHINE = "beaglebone"`
- Each machine should be described in the `README` file of the BSP.



Overview 2/2

- The hardware configuration file contains configuration variables related to the architecture and to the machine features.
- Some other variables help customize the kernel image or the filesystems used.



Machine configuration

- TARGET_ARCH: The architecture of the device being built.
- PREFERRED_PROVIDER_`virtual/kernel`: The default kernel.
- MACHINE_FEATURES: List of hardware features provided by the machine, e.g. `usbgadget usbhost screen wifi`
- SERIAL_CONSOLES: Speed and device for the serial consoles to attach. Used to configure `getty`, e.g. `115200;ttyS0`
- KERNEL_IMAGETYPE: The type of kernel image to build, e.g. `zImage`



MACHINE_FEATURES

- Lists the hardware features provided by the machine.
- These features are used by package recipes to enable or disable functionalities.
- Some packages are automatically added to the resulting root filesystem depending on the feature list.
  - The machine feature `keyboard` adds the `keymaps` to the image.



conf/machine/beaglebone-black.conf

```
#@TYPE: Machine
#@NAME: Beaglebone black machine
#@DESCRIPTION: Reference machine configuration for the http://beagleboard.org/black board

include conf/machine/include/arm/armv7a/tune-cortexa8.inc

IMAGE_FSTYPES = "tar.zst wic.zst wic.bmap"
WKS_FILE = "beaglebone-yocto.wks"
EXTRA_IMAGEDEPENDS += "virtual/bootloader"

SERIAL_CONSOLES ?= "115200;ttyS0"

PREFERRED_PROVIDER_virtual/kernel ?= "linux-yocto"
PREFERRED_VERSION_linux-yocto ?= "6.18%"
PREFERRED_PROVIDER_virtual/bootloader ?= "u-boot"

KERNEL_IMAGETYPE = "zImage"
KERNEL_DEVICETREE = "am335x-boneblack.dtb"
```



conf/machine/beaglebone-black.conf (cont.)

```
MACHINE_ESSENTIAL_EXTRA_RDEPENDS += "kernel-image kernel-devicetree"
MACHINE_EXTRA_RRECOMMENDS = "kernel-modules"

SPL_BINARY = "MLO"
UBOOT_SUFFIX = "img"
UBOOT_MACHINE = "am335x_evm_defconfig"

MACHINE_FEATURES = "usbgadget usbhost vfat alsa"

IMAGE_BOOT_FILES ?= "u-boot.${UBOOT_SUFFIX} ${SPL_BINARY} ${KERNEL_IMAGETYPE}
${KERNEL_DEVICETREE}"
```

- Original file: meta-yocto/meta-yocto-bsp/conf/machine/beaglebone-yocto.conf



Bootloader



Default bootloader 1/2

- By default (on ARM) the bootloader used is the mainline version of `U-Boot`, with a fixed version (per Yocto release).
- All the magic is done in `meta/recipes-bsp/u-boot/u-boot.inc`
- Some configuration variables used by the U-Boot recipe can be customized, in the machine file.



Default bootloader 2/2

- SPL_BINARY: If an SPL is built, describes the name of the output binary. Defaults to an empty string.
- UBOOT_SUFFIX: `bin` (default) or `img`.
- UBOOT_MACHINE: The target used to build the configuration.
- UBOOT_ENTRYPOINT: The bootloader entry point.
- UBOOT_LOADADDRESS: The bootloader load address.
- UBOOT_MAKE_TARGET: Make target when building the bootloader. Defaults to `all`.



Customize the bootloader

- It is possible to support a custom U-Boot by creating an extended recipe and to append extra metadata to the original one.
- This works well when using a mainline version of U-Boot.
- Otherwise it is possible to create a custom recipe.
  - Try to still use `meta/recipes-bsp/u-boot/u-boot.inc`



Kernel



Linux kernel recipes in Yocto

- There are mainly two ways of compiling a kernel:
  - By creating a custom kernel recipe, inheriting `kernel.bbclass`
  - By using the `linux-yocto` packages, provided in openembedded-core, for very complex needs
- The kernel used is selected in the machine file thanks to: PREFERRED_PROVIDER_`virtual/kernel`
- Its version is defined with: PREFERRED_VERSION_`<kernel_provider>`



Linux Yocto 1/3

- `linux-yocto` is a set of recipes with advanced features to build a mainline kernel
- `PREFERRED_PROVIDER_virtual/kernel = "linux-yocto"`
- `PREFERRED_VERSION_linux-yocto = "5.14%"`



Linux Yocto 2/3

- Automatically applies configuration fragments listed in SRC_URI with a `.cfg` extension

```
SRC_URI += "file://defconfig \
            file://nand-support.cfg \
            file://ethernet-support.cfg"
```



Linux Yocto 3/3

- Another way of configuring `linux-yocto` is by using *Advanced Metadata*.
- It is a powerful way of splitting the configuration and the patches into several pieces.
- It is designed to provide a very configurable kernel, at the cost of higher complexity.
- The full documentation can be found at https://docs.yoctoproject.org/kernel-dev/advanced.html#working-with-advanced-metadata-yocto-kernel-cache



Linux Yocto: Kernel Metadata 1/2

- Kernel Metadata is a way to organize and to split the kernel configuration and patches in little pieces each providing support for one feature.
- Two main configuration variables help taking advantage of this:
  - LINUX_KERNEL_TYPE: `standard` (default), `tiny` or `preempt-rt`
    - `standard`: generic Linux kernel policy.
    - `tiny`: bare minimum configuration, for small kernels.
    - `preempt-rt`: applies the `PREEMPT_RT` patch.
  - KERNEL_FEATURES: List of features to enable. Features are sets of patches and configuration fragments.



Linux Yocto: Kernel Metadata 2/2

- Kernel Metadata description files have their own syntax to describe an optional kernel feature
- A basic feature is defined as a patch to apply and a configuration fragment to add
- Simple example, `features/nunchuk.scc`

```
define KFEATURE_DESCRIPTION "Enable Nunchuk driver"

kconf hardware enable-nunchuk-driver.cfg
patch Add-nunchuk-driver.patch
```

- To integrate the feature into the kernel image:
  `KERNEL_FEATURES += "features/nunchuk.scc"`

Practical lab - Create a custom machine configuration



- Write a machine configuration
- Understand how the target architecture is chosen

Distro Layers

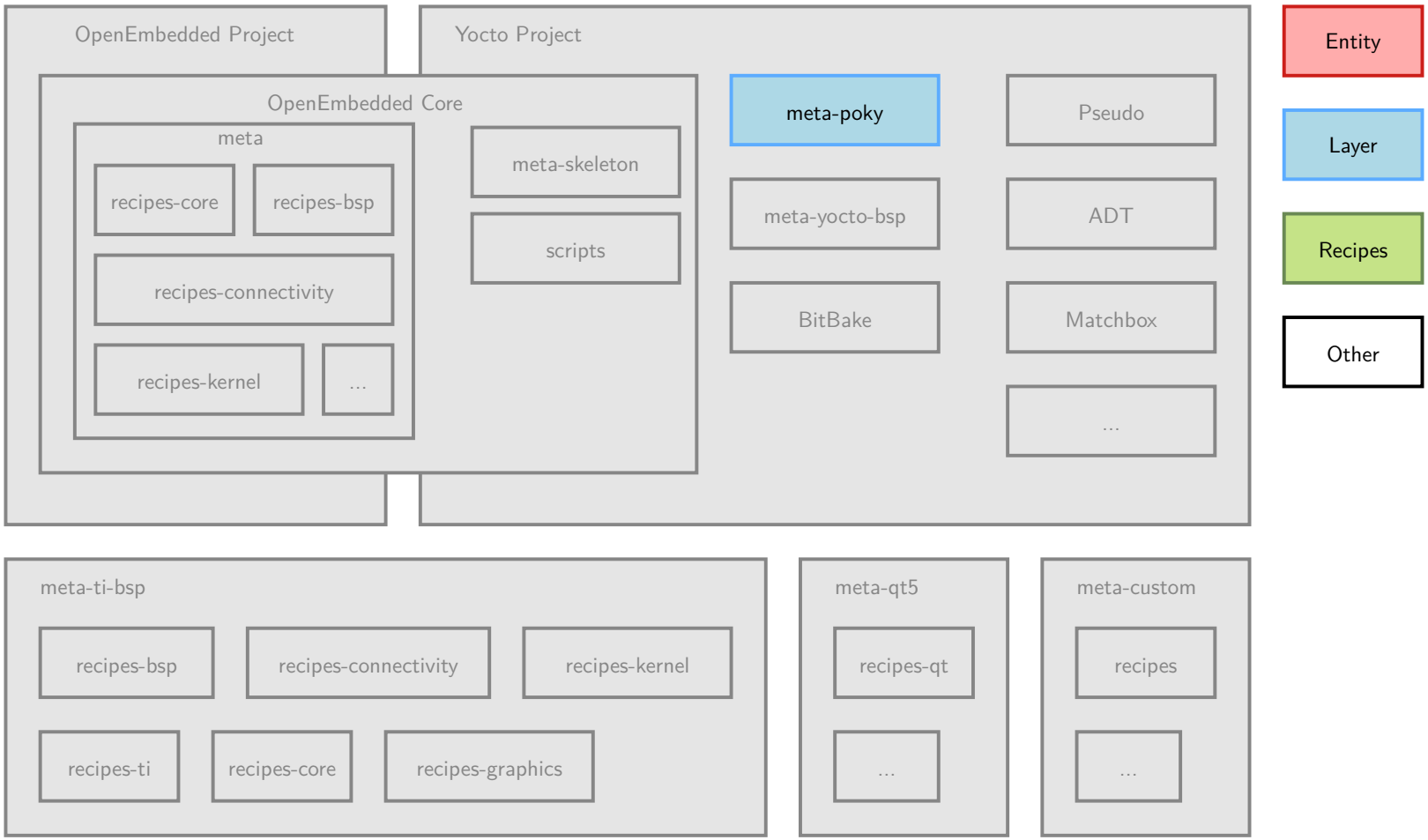


© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Distro Layers

Distro layers





Distro layers

- You can create a new distribution by using a Distro layer.
- This corresponds to the settings that have an impact on your packages. You can also decide to use Musl or Glibc, Wayland or X11, systemd or sysvinit...
- A distribution layer allows to change the defaults that are provided by `openembedded-core` or `poky`.
- It is useful to distribute changes that have been made in `local.conf`
- Note: Poky is a rather bloated distribution, mainly meant to be used for testing. It's not necessarily a good starting point to optimize the root filesystem for your own platform.



Best practice

- A distro layer is used to provide policy configurations for a custom distribution.
- It is a best practice to separate the distro layer from the custom layers you may create and use.
- It often contains:
  - Distro configuration files.
  - Specific classes (for example to sign images)
  - Distribution specific recipes: initialization scripts, splash screen...



Creating a Distro configuration file

- The configuration file for the distro layer is `conf/distro/<distro>.conf`
- This file must define the DISTRO_NAME variable.
- You can also use all the `DISTRO_*` variables.
- Use `DISTRO = "<distro>"` in `local.conf` to use your distro configuration.

```
DISTRO_NAME = "My Custom Distro"
DISTRO_VERSION = "1.0"
MAINTAINER = "..."

DISTRO_FEATURES = "sysvinit ipv4 ipv6 wifi zeroconf usbgadget usbhost pni-names"
```



DISTRO_FEATURES

- Lists the features the distribution will enable (SSL, WiFi, Bluetooth...).
- As for MACHINE_FEATURES, this is used by package recipes to enable or disable functionalities.
- For example, the `bluetooth` feature:
  - Asks the `bluez` daemon to be built and added to the image.
  - Enables bluetooth support in `ConnMan`.
- COMBINED_FEATURES provides the list of features that are enabled in both MACHINE_FEATURES and DISTRO_FEATURES.



Toolchain selection

- The toolchain selection is controlled by the TCMODE variable.
- It defaults to `"default"`.
- The `conf/distro/include/tcmode-${TCMODE}.inc` file is included.
  - This configures the toolchain to use by defining preferred providers and versions for recipes such as `gcc`, `binutils`, `*libc`...
- The providers' recipes define how to compile or/and install the toolchain.
- Toolchains can be built by the build system or external (rarely used because toolchains are fast to rebuild thanks to the shared state cache).



Sample files

- A distro layer often contains `sample files`, used as templates to build key configurations files.
- Example of `sample files`:
  - `bblayers.conf.sample`
  - `local.conf.sample`
- In `Poky`, they are in `meta-poky/conf/templates/default/`.
- The TEMPLATECONF environment variable controls where to find the samples and can be exported before sourcing `oe-init-build-env`.
- `##OEROOT##` can be used in `bblayers.conf.sample` and will be replaced by the path to the directory containing the `oe-init-build-env` script, after `oe-init-build-env` is sourced.
- `bitbake-layers save-build-conf` can be used to save the current configuration.